

Day 1 — PLC Training

Ladder Logic Foundations | 8:30 AM – 12:30 PM

Day 1 Syllabus

Duration: 225 min instructional + 15 min break

Prerequisites: None assumed. Designed for mechanical students with no prior PLC or programming exposure.

Software / Hardware Needed

- OpenPLC Editor installed on each student's laptop (or shared lab systems)
- Projector for live demos
- One physical PLC unit for hardware walkthrough (if available, optional)
- Printed PLC hardware diagram (1 per student, for labeling exercise)

Learning Objectives

By end of Day 1, students should be able to:

- 1 Explain what a PLC is and why it's used over manual/relay control
- 2 Identify the physical components of a PLC (CPU, PSU, input module, output module)
- 3 Explain the scan cycle in their own words
- 4 Wire and test a basic digital input and output
- 5 Explain the difference between NO and NC contacts and when each is used
- 6 Write a basic ladder rung using AND/OR logic
- 7 Build and test a complete Motor Start/Stop circuit with seal-in (latching)

Topic Breakdown

#	Topic	Time	Type
1	PLC Basics	20 min	Lecture
2	PLC Hardware	25 min	Lecture + demo + labeling exercise
3	PLC Scan Cycle	15 min	Lecture + whiteboard walkthrough
4	Inputs/Outputs + NO/NC Contacts	40 min	Hands-on wiring
—	Break	15 min	—
5	Ladder Logic Basics	45 min	Software walkthrough + guided exercise
6	Day Project: Motor Start/Stop (seal-in)	65 min	Independent/paired build

Assessment / Checkpoint

By end of day, every student/pair should have a working seal-in motor circuit running in OpenPLC Editor's simulator, demonstrated live to the instructor before leaving.

Homework / Pre-Day-2 Prep

None required — Day 2 builds directly on today's ladder basics.

Day 1 Learning Content

The teaching material — explanations, analogies, and worked examples for each topic.

1. PLC Basics (20 min)

What is a PLC? A PLC (Programmable Logic Controller) is an industrial computer that watches inputs (sensors, buttons, switches) and controls outputs (motors, lamps, valves) based on a program you write. Think of it as a very fast, very reliable decision-maker that never gets tired and never makes a wiring mistake once programmed correctly.

Why not just use relays? Before PLCs, factories used relay panels — physical switches wired together to make decisions. This worked, but changing the logic meant rewiring physical panels, panels took up huge cabinet space, and troubleshooting a fault meant physically tracing wires. A PLC replaces all that physical wiring with software logic — same decision-making, but you change it by editing a program instead of rewiring a cabinet.

Real-world examples to mention: conveyor belts starting/stopping based on sensors, automatic gates and boom barriers, water tank level control, packaging machines sequencing fill-seal-eject, elevators deciding which floor to stop at.

Instructor tip: Ask 2–3 students to name a machine they've personally seen and guess whether it's PLC-controlled. Gets them thinking in PLC terms before any technical content starts.

2. PLC Hardware (25 min)

A PLC has four core physical parts:

- **CPU (Central Processing Unit)** — the "brain." Runs the program you write, over and over, thousands of times a second.
- **Power Supply Unit (PSU)** — converts mains AC power (230V in India) into the low DC voltage (usually 24V) the PLC's internal circuits need.
- **Input Module** — where field devices connect: push buttons, sensors, switches, limit switches. Typically 24V DC signals. When a button is pressed, this module tells the CPU "this input is ON."
- **Output Module** — where the PLC controls field devices: motors (via contactors), lamps, solenoid valves, alarms. The CPU decides "turn this output ON," and the output module switches the power to the device.

Analogy for mechanical students: think of the CPU as the driver, inputs as the driver's eyes and ears (sensing the road), outputs as the driver's hands and feet (steering, braking). The PLC "drives" the machine the same way, just electronically and much faster.

Labeling exercise (5 min, part of this block): hand out a printed PLC diagram. Students label CPU, PSU, input terminals, output terminals. Walk around and correct as needed.

3. PLC Scan Cycle (15 min)

This is the single most important concept of the whole course — nearly every bug students hit later traces back to not understanding this.

The three-step loop, repeated forever:

- 1 **Input Scan** — PLC reads the current state of every input and stores it in memory.
- 2 **Logic Solve** — PLC runs through the entire ladder program, top to bottom, using the input values just read, and calculates what each output should be.
- 3 **Output Update** — PLC applies those calculated results to the actual physical outputs.

Then it repeats — instantly, continuously, often hundreds of times per second.

Key point to hammer home: during Step 2, the PLC does NOT re-check inputs mid-program even if a button state changes in real life. It only sees the snapshot from Step 1. This explains timing behavior that otherwise looks like a "bug."

Whiteboard walkthrough (part of this block): draw one simple rung (Button → Lamp) on the board. Walk through PLC reads button state → solves the rung → updates lamp. Do this twice in a row to show the repeating nature.

4. Inputs/Outputs + NO/NC Contacts (40 min)

Digital Input — a signal that is either ON (1) or OFF (0), like a button pressed or not pressed.

Digital Output — similarly ON or OFF: a lamp lit or not, a motor running or not.

NO (Normally Open) Contact — open (no connection) when nothing is pressing it. Pressing it closes the circuit, letting current flow. Default for most push buttons — "press to activate."

NC (Normally Closed) Contact — closed (current flowing) when nothing is pressing it. Pressing it opens the circuit, stopping current flow.

Why NC matters — the E-stop example: emergency stop buttons are almost always wired NC. If the wire to an NC button breaks or gets cut, the circuit opens automatically — the machine stops, fail-safe. If E-stops were wired NO instead, a cut wire would mean the E-stop silently stops working, which is dangerous. Worth spending real time on this — genuine safety reasoning, not just a technical detail.

Hands-on wiring (main block of this section):

- 1 Wire a push-button to a digital input terminal
- 2 Wire a lamp to a digital output terminal
- 3 In OpenPLC Editor's simulator, create a simple rung: Input → Output

- 4 Press the button, watch the lamp respond live on screen
- 5 Once working, have students physically identify which wire is the NO/NC path if using a real button with both contacts available

— BREAK — 15 min —

5. Ladder Logic Basics (45 min)

What ladder logic looks like: drawn like the rungs of a ladder — two vertical power rails on the left and right, with horizontal "rungs" of logic connecting them. Each rung represents one line of decision-making.

Basic symbols: normally open contact (input condition), normally closed contact (inverted input condition), output coil (what turns on/off).

Reading a rung left to right: if the conditions on the left are true, power "flows" through to the coil on the right, energizing it.

AND logic example:

```
-| Button A |—| Button B |—( Lamp )-
```

Both Button A AND Button B must be pressed for the Lamp to turn on — power has to flow through both to reach the coil.

OR logic example:

```
-| Button A |————( Lamp )-  
-| Button B |————■
```

Either Button A OR Button B being pressed turns the Lamp on — power has two possible paths.

Guided exercise (part of this block):

- 1 Build the AND example in OpenPLC Editor, test it — lamp should only light with both buttons pressed
- 2 Modify it into the OR example, test again — lamp should light with either button
- 3 Seeing the same two inputs behave completely differently based on series vs. parallel wiring is what makes the AND/OR distinction concrete

6. Day Project: Motor Start/Stop with Seal-In Circuit (65 min)

This is the day's capstone — the first fully independent build, combining everything covered today.

The problem to solve: a motor needs to start when a Start button is momentarily pressed, and keep running even after the button is released — until a Stop button is pressed. This "keep running after release" behavior needs a trick called a seal-in (or latching) circuit.

The logic, explained step by step:

- 1 Start button (NO) — momentary press, in series with:
- 2 Stop button (NC) — must NOT be pressed for the motor to run

- 3 Both feed into the Motor output coil
- 4 A contact from the Motor's own output is wired in parallel with the Start button — this is the seal-in. Once the motor turns on, this contact closes and "holds" the circuit on, even after the Start button is released.

Ladder diagram:

```
-| Start |———| Stop |—( Motor )—  
-| Motor |—■
```

(The bottom Motor contact is the seal-in — it's the same coil's own status feeding back as an input.)

Why this matters beyond this one project: this seal-in pattern is used everywhere in real industrial control — it's the foundation for almost every "run until told to stop" behavior. Once students genuinely understand this circuit, they've understood the core idea behind most real motor control panels they'll ever see.

Build sequence for students:

- 1 Wire physical Start button (NO), Stop button (NC), and Motor output (lamp standing in for motor is fine)
- 2 Build the rung in OpenPLC Editor exactly as described
- 3 Test: press Start briefly, release — motor should stay on
- 4 Test: press Stop — motor should turn off
- 5 Test: press Start and Stop at the same time — Stop should win (confirms the series wiring is correct)

Instructor note: expect most debugging time to go toward students forgetting the seal-in contact entirely (motor turns off the moment Start is released) or wiring Stop as NO instead of NC (motor won't run at all, or won't stop correctly). Both are common and worth letting students discover through testing rather than pointing out upfront — reinforces the concept much better than a fix instruction alone.

End of day checkpoint: every student/pair demonstrates a working seal-in circuit live before leaving.